

Numerische Fehleranalyse

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)

Material: Thomas Nagler

LMU München

Verwendete Vorkenntnisse

- Algorithmuskonzept
- Komplexitätsanalyse
- Verständnis, dass Algorithmen (meist) *approximative* Lösungen berechnen
- **Lineare Algebra I**
 - Vektor- und Matrixnormen
 - Normeigenschaften: Definitheit, Homogenität, Dreiecksungleichung
 - Matrixoperationen und Matrix-Vektor-Produkte
- **Analysis I**
 - Grenzwerte und Konvergenz
 - Stetigkeit
 - Grundlagen Taylor-Approximation

Fehleranalyse

Fehleranalyse

- Wenn ein Computer etwas speichert/rechnet, macht er Fehler.
- Der *absolute Fehler* ist dabei

$$\text{absoluter Fehler} = \text{fehlerhafter Wert} - \text{tatsächlicher Wert}.$$

- Es ist üblicher, Fehler *relativ* zu betrachten:

$$\text{relativer Fehler} = \frac{\text{fehlerhafter Wert} - \text{tatsächlicher Wert}}{\text{tatsächlicher Wert}}$$

- Vorteil: Keine Einheiten, Fehler in %.
- Nachteil: tatsächlicher Wert kann nicht 0 sein.
- Es gilt dann

$$\text{fehlerhafter Wert} = \text{tatsächlicher Wert} \times (1 + \text{relativer Fehler})$$

Fehlermaß

Def.: Fehlermaß

Sei V ein Vektorraum mit (Semi-)Norm $\|\cdot\|$ und $\tilde{v} \in V$ eine Approximation zu $v \in V$. Dann ist

- der **absolute Fehler**

$$\Delta_v = \tilde{v} - v,$$

- der **relative Fehler** (nur definiert falls $\|v\| > 0$)

$$\delta_v = \frac{\|\tilde{v} - v\|}{\|v\|} = \frac{\|\Delta_v\|}{\|v\|}.$$

- Ist $\|\Delta_v\| \leq \varepsilon$ bzw. $\delta_v \leq \varepsilon$, dann nennen wir ε eine absolute bzw. relative **Fehlerschranke**.

Fehlermaß

Lemma: Fehlerschranken

Es gilt

$$\|z\|(1 - \delta_z) \leq \|\tilde{z}\| \leq \|z\|(1 + \delta_z).$$

Beweis:

$$\delta_z = \frac{\|\tilde{z} - z\|}{\|z\|}$$

$$\|\tilde{z} - z\| = \delta_z \|z\|$$

$$||\tilde{z}\| - \|z\|| \leq \delta_z \|z\|$$

$$-\delta_z \|z\| \leq \|\tilde{z}\| - \|z\| \leq \delta_z \|z\|$$

$$\|z\| - \delta_z \|z\| \leq \|\tilde{z}\| \leq \|z\| + \delta_z \|z\|.$$

[umgekehrte Dreiecksungl.]

[Definition von Betrag]



Beispiel: Fehlermaße für Vektoren

Sei $\mathbf{v} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$ und $\tilde{\mathbf{v}} = \begin{pmatrix} 3.2 \\ 4.3 \end{pmatrix}$ mit der Euklidischen Norm $\|\cdot\|_2$.

- **Absoluter Fehler:**

$$\Delta_{\mathbf{v}} = \tilde{\mathbf{v}} - \mathbf{v} = \begin{pmatrix} 3.2 \\ 4.3 \end{pmatrix} - \begin{pmatrix} 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 0.2 \\ 0.3 \end{pmatrix}$$

- **Norm des absoluten Fehlers:**

$$\|\Delta_{\mathbf{v}}\|_2 = \sqrt{0.2^2 + 0.3^2} = \sqrt{0.04 + 0.09} = \sqrt{0.13} \approx 0.361$$

- **Relativer Fehler:**

$$\delta_{\mathbf{v}} = \frac{\|\Delta_{\mathbf{v}}\|_2}{\|\mathbf{v}\|_2} = \frac{\sqrt{0.13}}{\sqrt{3^2 + 4^2}} = \frac{\sqrt{0.13}}{5} \approx \frac{0.361}{5} \approx 0.072 = 7.2\%$$

- **Fehlerschranke:** Mit $\varepsilon = 0.1$ gilt $\delta_{\mathbf{v}} \approx 0.072 < 0.1$, also ist $\varepsilon = 0.1$ eine relative Fehlerschranke.

Fehlerzerlegung

- Sei
 - f ein Problem mit idealem Input $\boldsymbol{x}$
 - $\tilde{f}$ ein zugehöriger Algorithmus
 - und $\tilde{\boldsymbol{x}}$ der tatsächliche Input.
- Ein Algorithmus folgt dem Schema:
 - Input $\tilde{\boldsymbol{x}} \longrightarrow$ Algorithmus $\tilde{f} \longrightarrow$ Ergebnis $\tilde{f}(\tilde{\boldsymbol{x}})$
- Ein Fehler im Ergebnis $\tilde{f}(\tilde{\boldsymbol{x}}) \neq f(\boldsymbol{x})$ kann entstehen durch
 - Fehler im Input $\tilde{\boldsymbol{x}}$ (durch Messen/Speichern von $\boldsymbol{x}$)
 - unvermeidbar
 - Fehler im Algorithmus $\tilde{f}$ (durch Rundung/Approximation)
 - können wir beeinflussen

Fehlerzerlegung

$$\begin{aligned}\text{Gesamtfehler} &= \tilde{f}(\tilde{x}) - f(x) \\ &= \underbrace{\tilde{f}(\tilde{x}) - f(\tilde{x})}_{\substack{\text{Fehler im Algorithmus} \\ \text{(Maschinenarithmetik)}}} + \underbrace{f(\tilde{x}) - f(x)}_{\substack{\text{Folgefehler aus Input} \\ \text{(exakte Arithmetik)}}}\end{aligned}$$

- Erster Term misst nur den Fehler des Algorithmus (bei gegebenem/fixen Input), nicht den Inputfehler.

Ist er “klein”, nennen wir den Algorithmus $\tilde{f}$ **stabil**.

- Zweiter Term hängt nicht vom Algorithmus $\tilde{f}$ ab.

Ist er “klein”, nennen wir das Problem f an der Stelle x **gut konditioniert**.

Beispiel: Fehlerzerlegung

- Es gilt $e^x := \sum_{n=0}^{\infty} \frac{x^n}{n!}$
- Angenommen, wir wollen $f(\pi) = e^\pi$ berechnen und verwenden dazu die Approximationen $\tilde{\pi} = 3$ und $\tilde{f}(x) = \sum_{n=0}^2 x^n/n!$

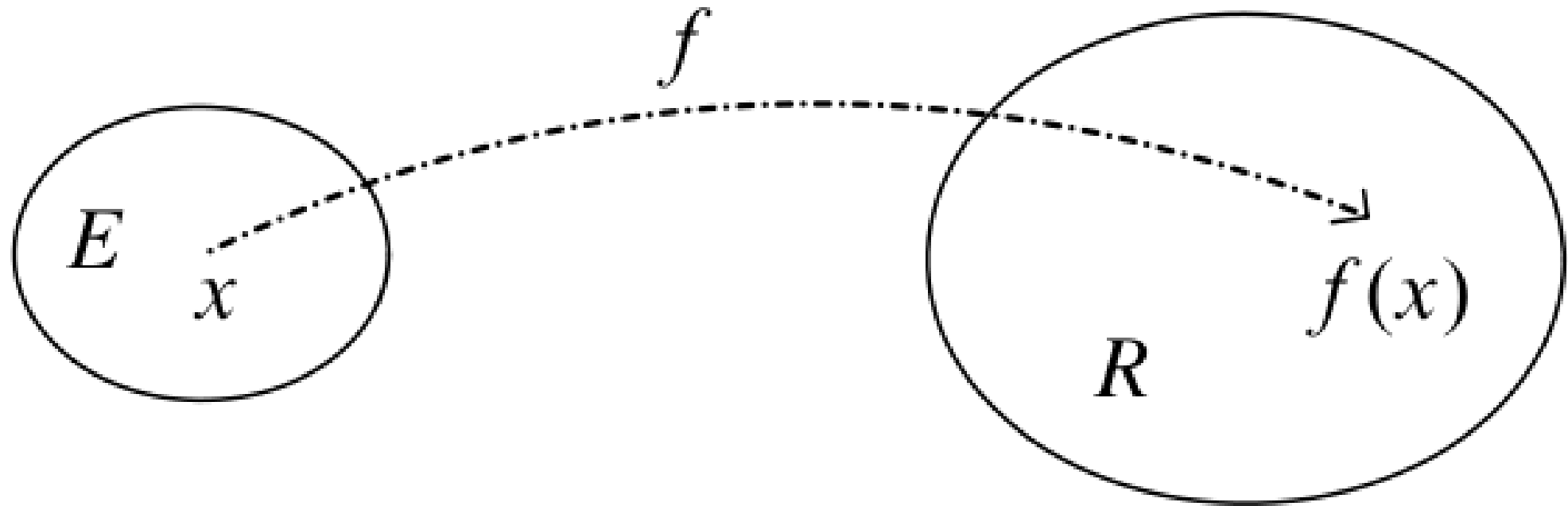
Zerlege den Gesamtfehler in die zwei erwähnten Teile.

Lösung:

$$\begin{aligned}\tilde{f}(3) - f(\pi) &= [\tilde{f}(3) - f(3)] + [f(3) - f(\pi)] \\ &= \text{Fehler im Algorithmus} + \text{Folgefehler aus Input} \\ &= \left[-\sum_{n=3}^{\infty} 3^n/n! \right] + [e^3 - e^\pi] \\ &\approx \left[\left(1 + \frac{3}{1} + \frac{9}{2}\right) - e^3 \right] + [20.1 - 23.1] \\ &\approx [-11.6] + [-3]\end{aligned}$$

Kondition

Kondition



- Inputfehler betrachten wir als *unvermeidbar*.
- Manche Probleme sind schwer zu lösen, weil sie *zu sensitiv bzgl. des Inputs* sind:
→ Outputmenge $R = f(E)$ ist *groß* gegenüber der Menge fehlerhafter Inputs
$$E = \left\{ \tilde{x} : \frac{\|\tilde{x} - x\|}{\|x\|} \leq \delta \right\}.$$
- Wir nennen das Problem dann **schlecht konditioniert**.

Kondition: Beispiel

Beispiel:

- Sei $f(x) = x^{-1}$
- Sei $x = 10^{-17}$, $\tilde{x} = x + \varepsilon$ mit $\varepsilon \in (-10^{-17}, 10^{-17})$.
- $\implies f(\tilde{x}) \in (1/2 \cdot 10^{17}, \infty)$ und

$$\left| \frac{f(\tilde{x}) - f(x)}{f(x)} \right| \in [0, \infty).$$

Achtung:

Die Störung ist hier *relativ zu x* nicht klein (bis zu 100%).

Bei relativ *kleinen* Störungen ist $x \mapsto 1/x$ dagegen harmlos: der relative Outputfehler ist dann ungefähr so groß wie der relative Inputfehler.

Es kommt also darauf an, *welche* Art von Inputfehler unvermeidbar ist.

Kondition

- Im Folgenden betrachten wir Abbildungen $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$.
- $\|\cdot\|$ bezeichnet dabei eine beliebige Vektornorm (bzw. die dadurch induzierte **Operatornorm** für Matrizen).

Def.: Konditionszahl

Sei f ein Problem mit Input $\mathbf{x}$.

- Die **absolute Konditionszahl** ist die kleinste Zahl $\kappa_{abs} = \kappa_{abs}(f, \mathbf{x})$ mit

$$\|f(\tilde{\mathbf{x}}) - f(\mathbf{x})\| \leq \kappa_{abs} \|\tilde{\mathbf{x}} - \mathbf{x}\|, \quad \text{für } \tilde{\mathbf{x}} \rightarrow \mathbf{x}.$$

- Die **relative Konditionszahl** (für $\|\mathbf{x}\| > 0$, $\|f(\mathbf{x})\| > 0$) ist die kleinste Zahl $\kappa_{rel} = \kappa_{rel}(f, \mathbf{x})$ mit

$$\frac{\|f(\tilde{\mathbf{x}}) - f(\mathbf{x})\|}{\|f(\mathbf{x})\|} \leq \kappa_{rel} \frac{\|\tilde{\mathbf{x}} - \mathbf{x}\|}{\|\mathbf{x}\|}, \quad \text{für } \tilde{\mathbf{x}} \rightarrow \mathbf{x}.$$

Interpretation

$$\|f(\tilde{\mathbf{x}}) - f(\mathbf{x})\| \leq \kappa_{abs} \|\tilde{\mathbf{x}} - \mathbf{x}\|, \quad \frac{\|f(\tilde{\mathbf{x}}) - f(\mathbf{x})\|}{\|f(\mathbf{x})\|} \leq \kappa_{rel} \frac{\|\tilde{\mathbf{x}} - \mathbf{x}\|}{\|\mathbf{x}\|}.$$

- *gut konditioniert* ($\kappa \lesssim 1$):
Inputfehler führt zu ähnlich großem oder kleinerem Outputfehler.
- *schlecht konditioniert* ($\kappa \gg 1$):
Inputfehler können zu viel größeren Outputfehlern führen.
- *schlecht gestellt* ($\kappa = \infty$):
Der Fehlerverstärkungsfaktor ist unbeschränkt — beliebig kleine Inputfehler können (relativ zu ihrer Größe) beliebig stark verstärkt werden.

Beispiel

- Sei $f(\boldsymbol{x}) = \boldsymbol{A}^{-1}\boldsymbol{x} = \boldsymbol{y}$.
- Absolute Konditionszahl:

$$\begin{aligned}\|f(\tilde{\boldsymbol{x}}) - f(\boldsymbol{x})\| &= \|\boldsymbol{A}^{-1}(\tilde{\boldsymbol{x}} - \boldsymbol{x})\| \leq \|\boldsymbol{A}^{-1}\| \|\tilde{\boldsymbol{x}} - \boldsymbol{x}\| \\ \implies \kappa_{abs} &= \|\boldsymbol{A}^{-1}\|\end{aligned}$$

- Relative Konditionszahl: Sei $\boldsymbol{h} = \tilde{\boldsymbol{x}} - \boldsymbol{x}$, dann

$$\begin{aligned}\frac{\|\boldsymbol{A}^{-1}\boldsymbol{h}\| / \|\boldsymbol{A}^{-1}\boldsymbol{x}\|}{\|\boldsymbol{h}\| / \|\boldsymbol{x}\|} &= \underbrace{\frac{\|\boldsymbol{A}^{-1}\boldsymbol{h}\|}{\|\boldsymbol{h}\|}}_{\leq \|\boldsymbol{A}^{-1}\|} \cdot \frac{\|\boldsymbol{x}\|}{\|\boldsymbol{A}^{-1}\boldsymbol{x}\|} \\ \implies \kappa_{rel} &= \|\boldsymbol{A}^{-1}\| \frac{\|\boldsymbol{x}\|}{\|\boldsymbol{A}^{-1}\boldsymbol{x}\|} = \kappa_{abs} \frac{\|\boldsymbol{A}\boldsymbol{y}\|}{\|\boldsymbol{y}\|} \leq \kappa_{abs} \|\boldsymbol{A}\| = \|\boldsymbol{A}^{-1}\| \|\boldsymbol{A}\|\end{aligned}$$

(Gleichheit im ersten Faktor, weil $\boldsymbol{h}$ entlang einer die Operatornorm maximierenden Richtung gewählt werden darf.)

- $\kappa(\boldsymbol{A}) = \|\boldsymbol{A}^{-1}\| \|\boldsymbol{A}\|$ wird in der linearen Algebra auch **Konditionszahl von $\boldsymbol{A}$** genannt – sie ist die von $\boldsymbol{x}$ unabhängige *obere Schranke* für κ_{rel} , nicht κ_{rel} selbst.
- Für die Operatornorm $\|\cdot\|_2$ (**Spektralnorm**) gilt

$$\kappa_2(\boldsymbol{A}) = \frac{\sqrt{\lambda_{\max}(\boldsymbol{A}^\top \boldsymbol{A})}}{\sqrt{\lambda_{\min}(\boldsymbol{A}^\top \boldsymbol{A})}}.$$

Quiz

Sei

$$\mathbf{A} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 0.1 \end{pmatrix}.$$

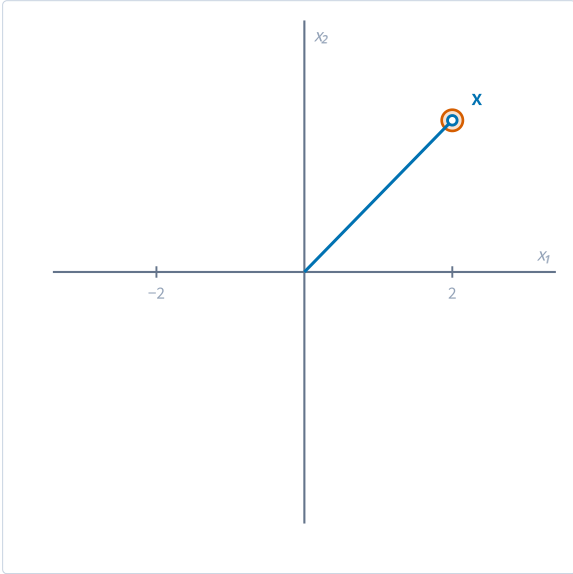
Was ist die Konditionszahl der Matrix bezüglich der 2-Norm?

- 1. 2
- 2. 20
- 3. 5
- 4. 50

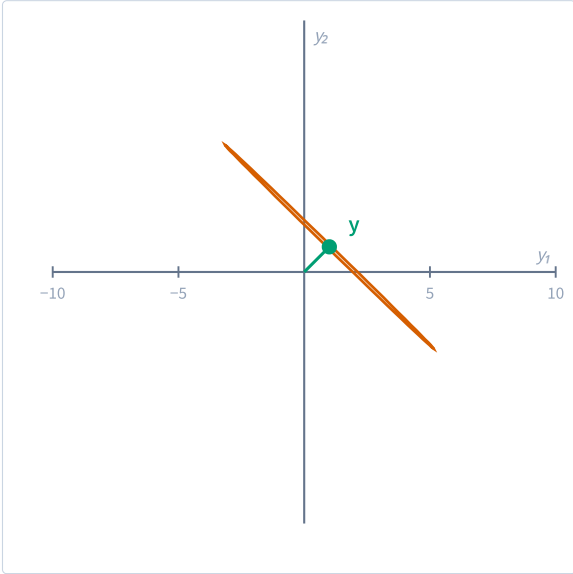
Interaktive Demo: Kondition

► Ziehen wir $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$ im linken Feld im Kreis herum und beobachten, wie sich die rote Ellipse rechts dabei streckt und staucht.

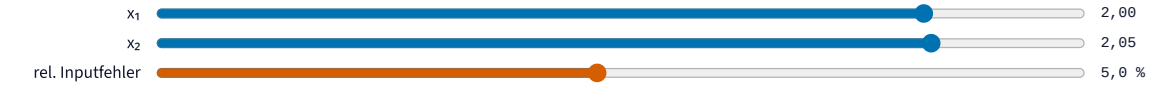
Inputebene: rechte Seite $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$ und die plausiblen Inputs



Lösungsebene: $\begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ und das Bild des Kreises



Drehung ($\kappa = 1$) mäßig gedehnt **schlecht konditioniert** fast singular



rel. Inputfehler

rel. Outputfehler

$A = \begin{pmatrix} 1,00 & 1,00; & 1,00 & 1,05 \end{pmatrix}$
 $y = A^{-1}x = \begin{pmatrix} 1,00; & 1,00 \end{pmatrix}$
Schranke $\kappa(A) = \sigma_{\max}/\sigma_{\min} = 82,04$ · ausgeschöpft zu 100 %

$\sigma_{\max} = 2,025, \sigma_{\min} = 0,0247$
 $\kappa_{\text{rel}}(f, x) = 82,03$

✗ **Die ungünstige rechte Seite.** $\kappa_{\text{rel}}(f, \begin{pmatrix} 1 \\ 0 \end{pmatrix}) = 82,0$ schöpft 100 % der Schranke $\kappa(A) = 82,0$ aus. Bemerkung 4.2.7 sagt genau das: Die Schranke ist kein Pessimismus, sie wird für ungünstige rechte Seiten angenommen. Aus 5,0 % Inputfehler werden hier 410 % Outputfehler.

Aufgabe

Sei $f(\mathbf{x}) = x_1 + x_2 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}^\top \mathbf{x} = \mathbf{1}^\top \mathbf{x}$.

1. Was sind die Konditionszahlen?
2. Für welche $\mathbf{x}$ ist das Problem schlecht konditioniert?

Lösung:

- Bezüglich der euklidischen Norm gilt (Cauchy-Schwarz)

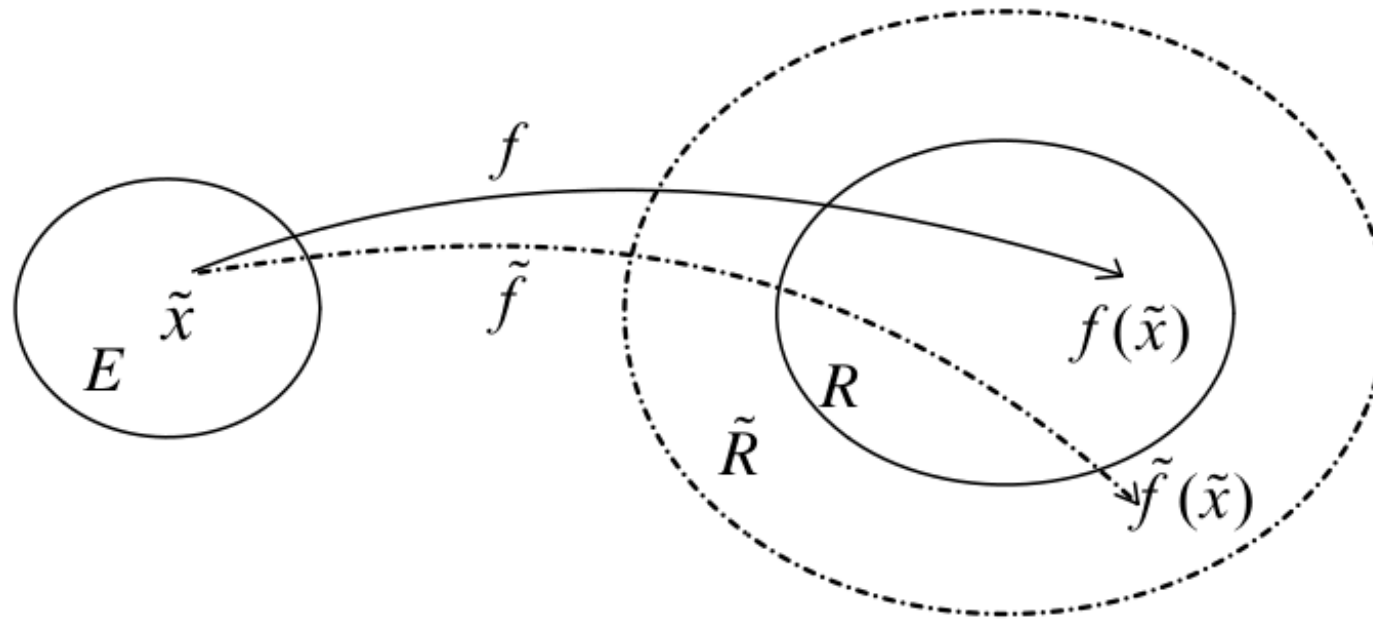
$$|f(\tilde{\mathbf{x}}) - f(\mathbf{x})| = |\mathbf{1}^\top (\tilde{\mathbf{x}} - \mathbf{x})| \leq \|\mathbf{1}\|_2 \|\tilde{\mathbf{x}} - \mathbf{x}\|_2 = \sqrt{2} \|\tilde{\mathbf{x}} - \mathbf{x}\|_2,$$

mit Gleichheit falls $(\tilde{\mathbf{x}} - \mathbf{x}) \parallel \mathbf{1}$, deshalb $\kappa_{abs} = \sqrt{2}$ und

$$\kappa_{rel} = \kappa_{abs} \frac{\|\mathbf{x}\|_2}{|x_1 + x_2|} = \sqrt{2} \frac{\|\mathbf{x}\|_2}{|x_1 + x_2|}.$$

- Für $0 < |x_1 + x_2| \ll \|\mathbf{x}\|_2$ ist $\kappa_{rel} \gg 1$ und das Problem somit schlecht konditioniert

Stabilität von Algorithmen



$$\begin{aligned}\text{Gesamtfehler} &= \tilde{f}(\tilde{x}) - f(x) \\ &= \underbrace{\tilde{f}(\tilde{x}) - f(\tilde{x})}_{\text{Fehler im Algorithmus}} + \underbrace{f(\tilde{x}) - f(x)}_{\text{Folgefehler aus Input}}\end{aligned}$$

- Analog zur Kondition können wir auch den algorithmischen Fehler $\tilde{f}(\tilde{x}) - f(\tilde{x})$ untersuchen.
- Ist er “klein”, nennen wir $\tilde{f}$ **stabil**.

Beispiel: Stochastic Gradient Descent

Optimierung der Verlustfunktion $L(\theta)$ mit $\theta_{k+1} = \theta_k - \alpha \hat{\nabla} L(\theta_k)$

Stabiler Algorithmus:

- Kleine Lernrate α (z.B. 0.001)
- (Numerische) Fehler akkumulieren langsam
- $\rightarrow$ Konvergiert zuverlässig zu einem Optimum

Instabiler Algorithmus:

- Zu große Lernrate α (z.B. 1.0)
- Verstärkt Rundung-/Schätzfehler des Gradienten $\hat{\nabla} L(\theta_k)$
- $\rightarrow$ Oszilliert oder divergiert

Das ist eine *Analogie* zum Konzept der **Stabilität** — hier mischen sich numerische Fehler und die Konvergenzdynamik des Verfahrens:

- Problem f : Finde Minimum von $L(\theta)$
- Algorithmus $\tilde{f}$: Stochastic Gradient Descent mit endlicher Präzision
- Algorithmischer Fehler: Akkumuliert Rundung-/Schätzfehler über k

Stabilität: Wichtigste Erkenntnis

Theorem: Stabilität zusammengesetzter Algorithmen

Sei f ein Problem mit Input $\mathbf{x}$, wobei $f(\mathbf{x}) = (h \circ g)(\mathbf{x})$ für zwei Teilprobleme g und h .
Dann gilt für einen Algorithmus $\tilde{f} = \tilde{h} \circ \tilde{g}$:

$$\frac{\|\tilde{f}(\mathbf{x}) - f(\mathbf{x})\|}{\|f(\mathbf{x})\|} \leq \frac{\|\tilde{h}(\tilde{\mathbf{y}}) - h(\tilde{\mathbf{y}})\|}{\|h(\mathbf{y})\|} + \kappa_{rel}(h, g(\mathbf{x})) \frac{\|\tilde{g}(\mathbf{x}) - g(\mathbf{x})\|}{\|g(\mathbf{x})\|},$$

wobei $\mathbf{y} = g(\mathbf{x})$, $\tilde{\mathbf{y}} = \tilde{g}(\mathbf{x})$.

- Ein schlecht konditioniertes Teilproblem h (an der Stelle $\mathbf{y} = g(\mathbf{x})$) verstärkt Fehler in $\tilde{g}$, wäre also ein *Indiz* für die Instabilität von $\tilde{f}$.
- Deshalb sollten wir **schlecht konditionierte Schritte immer so früh wie möglich** im Algorithmus ausführen.

Stabilität: Wichtigste Erkenntnis

Beweis:

$$\begin{aligned}\frac{\|\tilde{f}(\mathbf{x}) - f(\mathbf{x})\|}{\|f(\mathbf{x})\|} &= \frac{\|\tilde{h}(\tilde{g}(\mathbf{x})) - h(g(\mathbf{x}))\|}{\|h(g(\mathbf{x}))\|} \\&= \frac{\|\tilde{h}(\tilde{g}(\mathbf{x})) - h(\tilde{g}(\mathbf{x})) + h(\tilde{g}(\mathbf{x})) - h(g(\mathbf{x}))\|}{\|h(g(\mathbf{x}))\|} \\&\leq \frac{\|\tilde{h}(\tilde{g}(\mathbf{x})) - h(\tilde{g}(\mathbf{x}))\|}{\|h(g(\mathbf{x}))\|} + \frac{\|h(\tilde{g}(\mathbf{x})) - h(g(\mathbf{x}))\|}{\|h(g(\mathbf{x}))\|} \\&\leq \frac{\|\tilde{h}(\tilde{\mathbf{y}}) - h(\tilde{\mathbf{y}})\|}{\|h(\mathbf{y})\|} + \kappa_{rel}(h, g(\mathbf{x})) \frac{\|\tilde{g}(\mathbf{x}) - g(\mathbf{x})\|}{\|g(\mathbf{x})\|}.\end{aligned}$$

Fehler von $\tilde{h}$ selbst + durch h verstärkter Fehler von $\tilde{g}$ –

Idee: Einfügen von $0 = -h(\tilde{g}(\mathbf{x})) + h(\tilde{g}(\mathbf{x}))$, dann Dreiecksungleichung.

Beispiel: Stabilität der Varianz

Problem: Berechne Stichprobenvarianz

$$f(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2.$$

Algorithmen:

$$\tilde{f}_1(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2, \quad \tilde{f}_2(\mathbf{x}) = \left(\frac{1}{n} \sum_{i=1}^n x_i^2 \right) - \bar{x}^2$$

- Bei $\tilde{f}_2$ berechnen wir zum Schluss die Differenz $h(a, b) = a - b$ mit

$$a = \frac{1}{n} \sum_{i=1}^n x_i^2 \quad \text{und} \quad b = \bar{x}^2.$$

Beispiel: Stabilität der Varianz

- Die Konditionszahl ist dabei

$$\kappa_{rel}(h, (a, b)) = \sqrt{2} \frac{\sqrt{a^2 + b^2}}{|a - b|}.$$

- Falls a, b groß sind, aber $a \approx b$ gilt, ist $\kappa_{rel}(h, (a, b))$ groß.
- In diesem Fall kann h also vorangegangene Fehler massiv verstärken!

```
1 set.seed(5)
2 x <- rnorm(10^3, mean = 0, sd = 1) + 10^10
3 mean((x - mean(x))^2)
```

```
[1] 1.02
```

```
1 mean(x^2) - mean(x)^2
```

```
[1] 16384
```

Übersicht

- **Absoluter Fehler**: $\Delta_v = \tilde{v} - v$
- **Relativer Fehler**: $\delta_v = \frac{\|\tilde{v} - v\|}{\|v\|}$
- **Fehlerzerlegung**:
Gesamtfehler = **algorithmische Fehler** ($\tilde{f}(\tilde{x}) - f(\tilde{x})$) + **Input-Folgefehler** ($f(\tilde{x}) - f(x)$)
- **Kondition**: Sensitivität eines Problems bzgl. **Inputfehlern**
- **Konditionszahlen**: Absolut und relativ ($\kappa_{abs}, \kappa_{rel}$)
- **Konditionszahl einer Matrix**: $\kappa(\mathbf{A}) = \|\mathbf{A}\| \|\mathbf{A}^{-1}\|$
- **Stabilität**:
Ein Algorithmus ist stabil, wenn der **algorithmische Fehler** klein ist.
- **Stabilität zusammengesetzter Algorithmen**:
Fehlerfortpflanzung über Algorithmus-Teilschritte

Wrap-up

Heute gelernt

- Die *Kondition* eines Problems beschreibt dessen **Sensitivität** bzgl. Inputfehlern.
- Die *Stabilität* eines Algorithmus beschreibt dessen **Robustheit** bzgl. Fehler.
- **Schlecht konditionierte Teilschritte möglichst früh ausführen!**

Nächste Vorlesung

- Numerische Lösung von linearen Gleichungssystemen (LGS)